

Strategy-Aware Parameter-Efficient Adaptation for LLM-based Auto-Bidding

Songyue Cai*
sonnycai4649@gmail.com
University of Electronic Science and
Technology of China
Chengdu, China
Taobao & Tmall Group of Alibaba
Beijing, China

Lianyu Wang
wanglianyu.wly@taobao.com
Taobao & Tmall Group of Alibaba
Beijing, China

Shan Gu
gushan.gs@taobao.com
Taobao & Tmall Group of Alibaba
Beijing, China

Ziru Xu
ziru.xzr@taobao.com
Taobao & Tmall Group of Alibaba
Beijing, China

Jian Xu
xiyu.xj@taobao.com
Taobao & Tmall Group of Alibaba
Beijing, China

Xiaofeng Zhu^{†‡}
zhuxf@hainanu.edu.cn
Hainan University
Haikou, China

Bo Zheng[†]
bozheng@alibaba-inc.com
Taobao & Tmall Group of Alibaba
Beijing, China

Abstract

Advertising bidding has evolved from manual strategies to auto-bidding systems better adapted for large-scale, dynamic auction environments. While recent advances in Large Language Models (LLMs) offer strong reasoning for auto-bidding, existing methods suffer from shallow trajectory-text interactions and require costly fine-tuning, hindering the efficient use of pretrained knowledge under diverse constraints. To address these challenges, we propose **SAGE**, a novel **S**trategy-aware **A**uto-bidding framework **G**uided by LLMs for **E**fficient bidding. SAGE introduces a parameter-efficient multi-modal alignment framework for constrained auto-bidding with LLMs. Specifically, SAGE comprises three key components: (i) the **position augmentation module** adopts temporal-semantic positional embeddings to effectively capture the intrinsic dynamics and semantic structures; (ii) the **text alignment module** leverages gated cross-attention to align the embedding spaces of trajectory and text modalities, enabling effective multi-modal fusion while alleviating the computational overhead caused by long trajectories; (iii) the **constraint-gated LoRA module** employs constraints as routing signals, activating only a small subset of experts to adapt the

behavior of a frozen LLM efficiently. Extensive experiments on large-scale auto-bidding benchmark demonstrate that SAGE consistently achieves superior performance while tuning less than **10%** of the trainable parameters required by full fine-tuning. Ablation studies further validate the critical contribution of each component to the framework’s overall performance. The source code will be released at <https://github.com/YuzunoKawori/SAGE>.

CCS Concepts

• Information systems → Computational advertising.

Keywords

Auto-bidding, Large Language Model, Generative Model, Efficient Tuning

1 Introduction

Advertising bidding is a key decision-making problem in online advertising systems, where the goal is to determine optimal bids for billions of daily impression opportunities in a dynamic auction environment to achieve diverse advertisers’ economic objectives. With the rapid digitalization of commerce in recent years, the sheer volume of traffic and intensity of competition [38] have rendered traditional manual bidding by human experts impractical [10, 50]. Auto-bidding systems have become an important solution to the limitations of traditional manual bidding in high-traffic and highly competitive environments. By leveraging trained models, such systems can generate precise bids for large-scale ad traffic in real time based on current states and environmental information. Specifically, auto-bidding treats advertisers’ bidding decisions as a constrained sequential decision-making problem, where the goal of auto-bidding is to dynamically adjust bidding parameters throughout the delivery period to maximize conversions

*Work is done during the internship at Taobao & Tmall Group of Alibaba

[†]Corresponding authors.

[‡]Xiaofeng Zhu is the master’s advisor of Songyue Cai.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

while satisfying advertiser-specified constraints such as cost-per-action (CPA) [18, 37]. As a result, auto-bidding has become a crucial mechanism for advertisers to achieve their business objectives in competitive advertising markets [43].

Existing auto-bidding methods can generally be divided into four categories, *i.e.*, offline reinforcement learning (RL) methods [24, 25, 37, 51], sequence modeling-based methods [4, 13, 26, 56], planning-based generative bidding methods [14, 16, 21, 28], and large language model (LLM)-based multi-modal methods [29, 36, 59]. Specifically, offline RL methods learn bidding policies from offline datasets by estimating state-action value functions and optimizing policies via Bellman backups. For example, CQL [25] performs conservative value estimation for out-of-distribution actions to mitigate Q-value overestimation in offline RL, thereby improving the stability of policy learning. However, accurately estimating value functions in long-horizon and sparse-reward bidding environments remains challenging, often leading to unstable policy learning. To mitigate these challenges, researchers have turned to sequence modeling paradigms. Sequence modeling-based generative methods reformulate auto-bidding as a sequence modeling task and directly generate actions by modeling historical trajectories without explicitly learning value functions [1]. For instance, decision transformer (DT) [4] models return-to-go (RTG), states, and actions using a transformer [48] architecture and autoregressively generates bidding actions conditioned on the desired return. Nevertheless, beyond trajectory generation, researchers have also explored explicit modeling of future dynamics and long-term planning in auto-bidding. Planning-based generative bidding methods further introduce planning mechanisms over the environment or future states on top of generative policies, guiding current bidding decisions by predicting future information. For example, DiffBid [16] learns temporal periodic distributions to perform strategy planning and then generates actions through a control module. Since offline RL methods and generative methods primarily operate on structured trajectory data, emerging LLM-based multi-modal methods further enable advertisers' constraints (*e.g.*, CPA) to be expressed through natural language. For example, LBM [29] adopts a hierarchical framework where an LLM first generates a planning chain-of-thought under given bidding objectives and then produces actions guided by plan.

Despite the recent progress of LLM-based multi-modal methods, existing methods still face a fundamental challenge in achieving effective multi-modal alignment and strategy-aware parameter-efficient adaptation under complex constraints. This challenge mainly manifests in three aspects. First, in the trajectory representation stage, existing methods typically employ simple learnable positional embeddings to model sequential trajectories. Such designs mainly capture temporal dependencies, but overlook the semantic differences among token types, thus weakening the model's ability to encode the structural information of bidding sequences. For example, DecisionLLM [36] assigns identical positional embeddings to different token types within the same timestep, making it difficult for the model to distinguish their semantic roles effectively. Second, in the multi-modal fusion stage, many methods rely on naive concatenation to fuse trajectory and text modalities before feeding them into the LLM, ignoring the embedding space discrepancy across modalities. This coarse-grained fusion strategy not only hinders effective cross-modal alignment but also incurs

additional computational overhead. For instance, LBM [29] directly concatenates trajectory, task description, and constraint for joint encoding, making it difficult to establish effective cross-modal interactions while also introducing excessively long input sequences. Finally, in the model adaptation stage, most existing methods rely on full fine-tuning to adapt LLMs to the auto-bidding task, requiring updates to a large number of parameters. This not only results in substantial training and deployment costs but also limits the model's transferability to new tasks. For example, both LBM [29] and DecisionLLM [36] adopt full fine-tuning strategies that update all parameters of the pretrained LLMs during training, resulting in substantial retraining costs when adapting to new tasks.

To address the above challenges, we propose a novel **Strategy-aware Auto-bidding framework Guided by LLMs for Efficient bidding**, referred to as **SAGE**. The overall framework is illustrated in Figure 1 and consists of three key modules designed to systematically bridge the gaps in representation, fusion, and adaptation. Firstly, the **position augmentation module** disentangles positional embeddings into temporal and semantic components and assigns different positional embeddings to tokens across different timestep and semantic types, thereby enhancing the model's understanding of trajectory structures while reducing additional parameters and addressing the first challenge. Secondly, the **text alignment module** performs efficient fusion between trajectory modality and text task description through cross-attention before encoding while aligning their embedding spaces, thereby improving the model's understanding of text instructions and significantly shortening the input length and addressing the second challenge. Finally, the **constraint-gated LoRA module** freezes the pretrained backbone and updates only a small subset of parameters for efficient fine-tuning, while utilizing task constraints as gating signals to select different experts, thereby enabling adaptive strategy selection with low computational overhead and addressing the third challenge. Compared with previous methods, the main contributions of our work are summarized as follows:

- We propose **SAGE**, a novel framework that makes LLM-based auto-bidding both strategy-aware and parameter-efficient, significantly improving decision-making with reduced computational overhead.
- We design the position augmentation module that enhances trajectory modeling by disentangling temporal and semantic positional information, enabling a deeper structural understanding with high parameter efficiency.
- We introduce the text alignment module that effectively aligns trajectory and text embeddings while significantly compressing the input for the LLM.
- We design the constraint-gated LoRA module that enables parameter-efficient fine-tuning for auto-bidding by utilizing condition constraint as gating signals to dynamically adapt bidding strategies with minimal trainable parameters.
- We demonstrate that SAGE achieves superior performance on a large-scale advertising bidding benchmark while using 91.70% fewer trainable parameters than full fine-tuning.

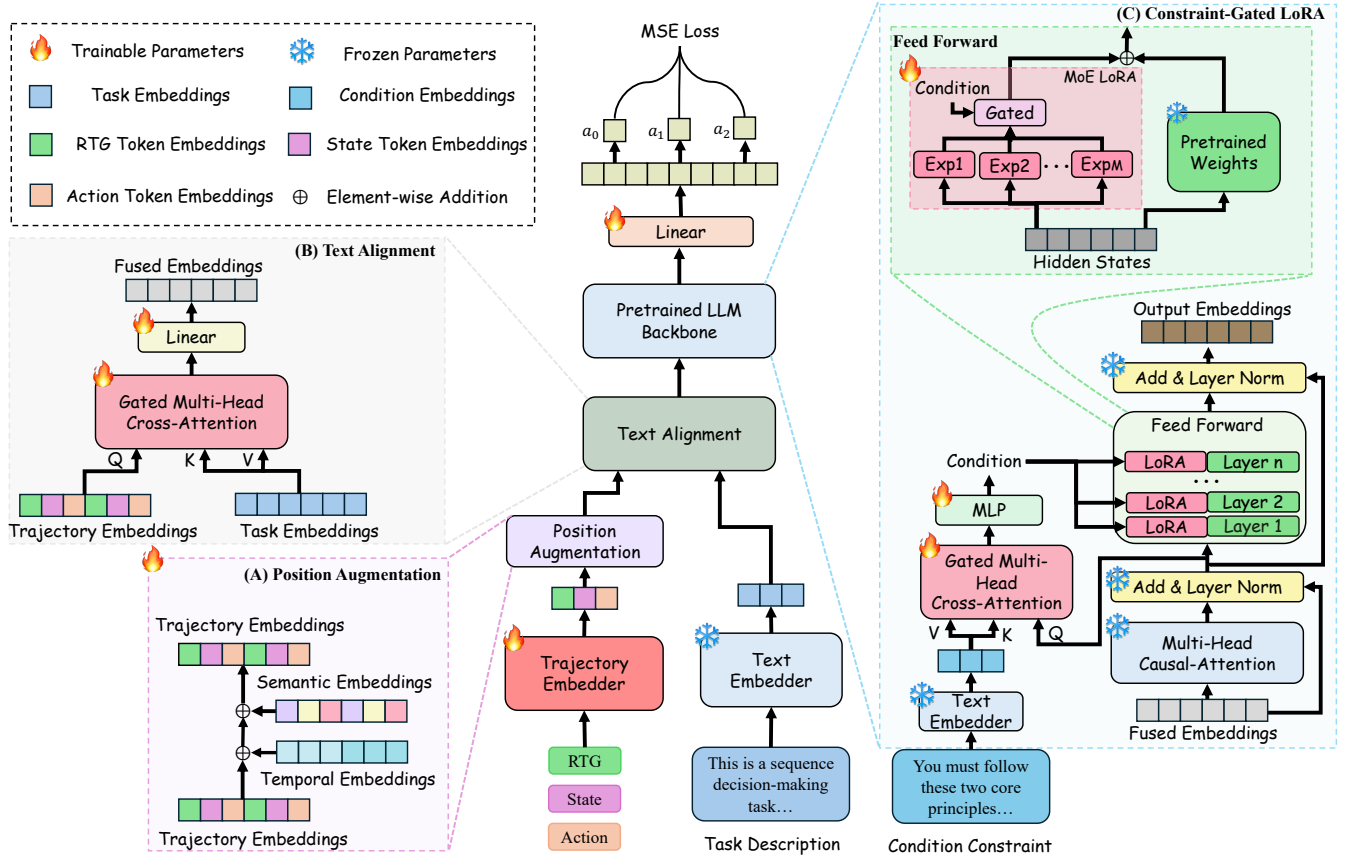


Figure 1: The overall flowchart of the proposed SAGE. SAGE consists of three key modules (A, B, and C). Given an advertising bidding trajectory, SAGE first uses a trainable trajectory embedder to obtain the trajectory embeddings. The module (A) adds temporal–semantic positional embeddings to the trajectory embeddings. The module (B) then fuses the trajectory embeddings with the text embeddings obtained by encoding the task description through the frozen embedding layer of the LLM, to generate the fused embeddings. The module (C) first feeds the fused embeddings into the frozen pretrained LLM backbone to obtain intermediate hidden states from the causal attention layers. It then integrates the hidden states with the condition embeddings encoded from the condition constraint using the same frozen LLM embedding layer, to generate the condition gating signal. Based on this signal, Mixture-of-Experts (MoE) LoRA performs parameter-efficient adaptation on the frozen LLM backbone. Finally, a linear layer projects the last hidden states to produce the predicted action at the current timestep.

2 Preliminaries

2.1 Problem Statement

In online advertising systems, thousands of advertisers compete for limited inventory by submitting bids over a time horizon comprising a chronological trajectory of D ad opportunities. For each opportunity $i \in \{1, \dots, D\}$, an advertiser submits a bid_i . During the auction process, the advertiser with the highest bid wins the impression, obtains the corresponding value v_i , and pays the associated cost c_i , which is typically determined by the second-price auction mechanism. The advertiser’s objective is to maximize the total value obtained from advertising impressions under a predefined budget constraint. In many practical scenarios, advertisers may also impose additional constraints, such as CPA [18]. Therefore, an effective auto-bidding strategy should maximize the advertiser’s overall utility while satisfying these constraints. Formally, the objective of the

auto-bidding problem can be expressed as:

$$\begin{aligned}
 & \max \quad \sum_i o_i v_i \\
 & \text{s.t.} \quad \sum_i o_i c_i \leq B \\
 & \quad \sum_i c_{ij} o_i \leq C_j, \quad \forall j \\
 & \quad o_i \in \{0, 1\}, \quad \forall i
 \end{aligned} \tag{1}$$

where o_i denotes whether the advertiser wins the i -th impression, v_i denotes the corresponding value, B denotes the advertiser’s budget, c_{ij} denotes cost associated with constraint j for impression i , C_j is the upper bound of j -th constraint provided by the advertiser, and p_{ij} denotes the performance indicator, e.g., conversions.

Previous studies have derived the theoretically optimal bidding formula under such constraints [18]:

$$bid_i^* = \lambda_0 v_i + \sum_{j=1}^J \lambda_j p_{ij} C_j, \quad (2)$$

where bid_i^* denotes the optimal bid for the i -th impression, λ_0 denotes the optimal bidding parameter, J denotes the number of constraints, and λ_j denotes the Lagrange multiplier for the j -th constraint. The objective is to dynamically adjust these parameters to optimize bidding decisions under multiple constraints.

2.2 Sequential Decision for Auto-Bidding

In dynamic auction marketplaces where both competitor strategies and opportunity distributions evolve continuously, static bidding parameters inevitably become suboptimal. This necessitates adaptive parameter adjustments based on real-time market feedback, fundamentally casting auto-bidding as a sequential decision-making problem. We formalize this process by defining the following interconnected components, where the model observes state s_t to select action a_t , receives immediate reward r_t as environmental feedback, and optimizes toward the cumulative RTG R_t :

- State s_t : a feature vector describing the bidding environment at timestep t , such as remaining budget, remaining time, and historical bidding information.
- Action a_t : the action taken at timestep t , represented as $a_t = (a_t^{\lambda_0}, a_t^{\lambda_1}, \dots, a_t^{\lambda_J})$.
- Reward r_t : the total value obtained from impressions won during timestep t . Given D_t candidate impressions in this period, the reward is defined as $r_t = \sum_{i=1}^{D_t} o_i v_i$, where D_t denotes the number of ad opportunities at the timestep t .
- RTG R_t : the cumulative reward from timestep t until the terminal timestep T , defined as $R_t = \sum_{n=t}^T r_n$.

With these definitions, the entire bidding procedure can be represented as a trajectory $\tau = (R_0, s_0, a_0, \dots, R_{T-1}, s_{T-1}, a_{T-1})$.

2.3 LLM for Auto-Bidding

LLM-based multi-modal methods have emerged as a promising direction for auto-bidding [29, 36], leveraging pretrained LLM and text prompts to enhance task understanding and enforce conditional constraints (e.g., CPA). Given a bidding decision trajectory τ , previous LLM-based multi-modal methods [29, 36] first employ a lightweight trainable embedder $\mathcal{S}(\cdot)$ to encode the trajectory modality and obtain its embedding $\mathbf{e} = \mathcal{S}(\tau) \in \mathbb{R}^{L \times d}$, where $L = 3T$ denotes the trajectory length and d denotes the hidden dimension of the LLM. The trajectory embeddings consist of a sequence of embeddings across timestep, where timestep t comprises three token types: RTG R_t , state s_t , and action a_t , formulated as:

$$\mathbf{e}_t = \mathcal{S}(R_t, s_t, a_t), \quad \mathbf{e} = (\mathbf{e}_0, \mathbf{e}_1, \dots, \mathbf{e}_{T-1}), \quad \mathbf{e}_t \in \mathbb{R}^{3 \times d}. \quad (3)$$

Temporal information is injected by adding learnable positional embeddings $\mathbf{pos} = (\mathbf{pos}_0, \dots, \mathbf{pos}_{T-1}) \in \mathbb{R}^{T \times d'}$ uniformly to all token types $k \in \{R, s, a\}$ within each timestep:

$$\mathbf{e}_{t,k} = \text{Linear}(\text{Concat}(\mathbf{e}_{t,k}, \mathbf{pos}_t)), \quad (4)$$

where $\mathbf{e}_{t,k} \in \mathbb{R}^d$ denotes the embedding of the token of type k at timestep t , d' denotes the dimension of positional embeddings,

$\text{Concat}(\cdot)$ denotes the concatenation operation. For the text modality, a text prompt p is encoded via the frozen pretrained LLM embedder $\mathcal{T}(\cdot)$ into text embeddings $\mathbf{e}_{\text{text}} = \mathcal{T}(p) \in \mathbb{R}^{N \times d}$, where N denotes the tokenized prompt length. Then, the text and trajectory embeddings are concatenated along the trajectory dimension to form the fused embeddings $\mathbf{x}$:

$$\mathbf{x} = \text{Concat}(\mathbf{e}_{\text{text}}, \mathbf{e}), \mathbf{x} \in \mathbb{R}^{(N+L) \times d}. \quad (5)$$

Next, $\mathbf{x}$ is encoded into the hidden state $\mathbf{H} = \text{Backbone}(\mathbf{x})$ by the LLM backbone, where $\mathbf{H} \in \mathbb{R}^{(N+L) \times d}$, from which the state token $\mathbf{h}_t^s \in \mathbb{R}^d$ is linearly projected to predict the action:

$$\hat{a}_t = \text{Linear}(\mathbf{h}_t^s). \quad (6)$$

Finally, the model is optimized via gradient descent using the mean squared error loss (MSE Loss):

$$\mathcal{L} = \frac{1}{T} \sum_{t=0}^{T-1} \|a_t - \hat{a}_t\|^2. \quad (7)$$

3 Methodology

3.1 Motivation

Based on the above formulation, we further analyze the inherent limitations of existing LLM-based auto-bidding methods from the perspectives of trajectory representation, multi-modal fusion, and model adaptation. Specifically, in trajectory representation, existing methods inject temporal information by uniformly adding learnable positional embeddings to all token types within the same timestep according to Eq. (4). Such a design implicitly assumes semantic homogeneity across RTG, state, and action tokens, making it difficult for the model to distinguish their distinct functional roles and thereby limiting its ability to capture the hierarchical structure of bidding trajectories. Moreover, the reliance on learnable embeddings introduces additional parameters and necessitates retraining when trajectory lengths vary, constraining transferability across different bidding horizons.

In multi-modal fusion, previous methods directly concatenate trajectory embeddings with text embeddings according to Eq. (5). This coarse-grained fusion strategy treats trajectory data and textual instructions as semantically aligned by default, overlooking the inherent embedding space discrepancy and the semantic hierarchy between task description and condition constraint. As a result, it fails to establish effective cross-modal interactions while also introducing additional text embeddings for the LLM backbone to encode, leading to inefficient multi-modal fusion.

In model adaptation, previous methods optimize the model by fully fine-tuning the LLM backbone according to Eq. (7). As a result, the rich pretrained prior knowledge may be diluted. This paradigm also incurs substantial computational costs and limits the model's adaptability when transferred to new advertising scenarios. These intertwined issues collectively motivate our core research question: *How can we achieve strategy-aware, parameter-efficient adaptation of LLMs for auto-bidding while preserving structural fidelity, cross-modal alignment, and pretrained knowledge?*

To address these limitations, we propose a novel LLM-based auto-bidding framework with three components: position augmentation, text alignment, and constraint-gated LoRA modules. The position

augmentation module (Section 3.2) tackles the entangled trajectory embeddings, the text alignment module (Section 3.3) resolves the inefficient cross-modal fusion, and the constraint-gated LoRA module (Section 3.4) overcomes the costly adaptation paradigm.

3.2 Position Augmentation

Previous methods typically design positional embeddings for trajectory tokens by only considering temporal differences across timestep, while overlooking that auto-bidding trajectories naturally contain multiple tokens with different semantics within the same timestep. Moreover, these methods usually adopt learnable positional embeddings by assigning a trainable parameter to each timestep. When the input trajectory length changes, such encodings often become ineffective.

Semantic embeddings. To address this issue, we design a position augmentation module. Specifically, a typical auto-bidding trajectory contains three tokens at each timestep: RTG R_t , state s_t , and action a_t . These tokens represent different semantic information and provide distinct perspectives for the model during decision-making. In addition, existing LLM-based methods often explicitly emphasize the names and semantics of different tokens when constructing text prompts [29]. Therefore, ignoring semantic differences when designing positional embeddings may lead to incomplete information representation and further weaken the LLM’s ability to understand auto-bidding trajectories. Based on this observation, it is necessary to design distinct positional embeddings for tokens with different semantics. To achieve this, we introduce only a small number of trainable parameters $\text{pos}^s = (\text{pos}_R^s, \text{pos}_s^s, \text{pos}_a^s)$, $\text{pos}^s \in \mathbb{R}^{3 \times d}$ to represent semantic embeddings for each token type, thereby enhancing the semantic information within the trajectory. Formally, this process can be described by the following equation:

$$\mathbf{e}_{t,k} = \mathbf{e}_{t,k} + \text{pos}_k^s, \quad k \in \{R, s, a\}. \quad (8)$$

Through Eq. (8), we assign consistent semantic embeddings to tokens with the same semantic type across different timestep, which enhances the distinction between different semantic tokens and improves the LLM’s understanding of trajectory information.

Temporal embeddings. Furthermore, we consider the design of temporal embeddings. Previous methods typically use a trainable parameter $\text{pos} \in \mathbb{R}^{T \times d}$ as the temporal embeddings. However, in real-world environments, bidding trajectories often have varying numbers of timestep. In such cases, this method not only introduces additional trainable parameters but also lacks effective scalability when the trajectory length changes. To address this issue, we adopt a non-parametric positional encoding scheme, namely sinusoidal positional encoding [48], to obtain temporal embeddings $\text{pos}' = (\text{pos}'_0, \text{pos}'_1, \dots, \text{pos}'_{T-1})$, $\text{pos}' \in \mathbb{R}^{T \times d}$. Finally, we replace the original positional embeddings with the proposed position augmentation module. Formally, it can be written as follows:

$$\mathbf{e}_{t,k} = \mathbf{e}_{t,k} + \text{pos}'_t + \text{pos}_k^s. \quad (9)$$

By replacing Eq. (4) with Eq. (9), we model the missing semantic positional information in previous methods with few trainable parameters, while adopting non-parametric temporal embeddings to improve scalability. By jointly modeling semantic and temporal information, the module enhances the distinction among tokens with different semantics and supports variable-length trajectories.

As a result, it improves the LLM’s understanding of trajectory information and addresses the first limitation of previous methods.

3.3 Text Alignment

In Section 3.2, we introduce a position augmentation module that enhances trajectory embeddings by incorporating semantic embeddings and improved temporal embeddings, enabling LLMs to better understand trajectory modality and facilitating multi-modal fusion with text information. However, existing methods typically rely on simple concatenation for multi-modal fusion, which fails to adequately capture complex interactions between trajectory and text modalities and makes it difficult to effectively align their heterogeneous feature spaces without full fine-tuning, potentially disrupting pretrained prior knowledge. At the same time, auto-bidding prompts often contain a detailed task description and condition constraint, substantially increasing the input trajectory length and introducing considerable computational overhead.

Motivated by these observations, we propose the text alignment module to achieve efficient and effective multi-modal fusion. Inspired by the success of cross-attention mechanisms [48] in information retrieval and alignment tasks [2, 15, 22, 27, 32, 44], we introduce a gated cross-attention where trajectory embeddings serve as the query and text prompt embeddings act as the key and value. In cross-attention, the query representation retrieves relevant semantic information from the key-value pairs while preserving the original sequence structure of the query modality, enabling flexible cross-modal interaction. Hence, using trajectory embeddings as queries allows the model to enhance trajectory representations with task semantics while maintaining compatibility with existing trajectory prediction paradigms. In addition, it is worth noting that the complex condition constraint is difficult to interpret directly at the embedding level. Therefore, the text prompt used in this module mainly describes the task semantics to enhance trajectory embeddings, while deeper interactions with constraint conditions will be handled in Section 3.4.

Accordingly, the proposed text alignment module aligns the trajectory and text modalities to obtain the fused embeddings. The formulation is given as follows:

$$\mathbf{x} = \text{Linear}(\text{Gated_attn}(Q = \mathbf{e}, K = \mathbf{e}_{\text{task}}, V = \mathbf{e}_{\text{task}})), \mathbf{x} \in \mathbb{R}^{L \times d}, \quad (10)$$

where Q , K , and V represent the query, key, and value embeddings, $\text{Gated_attn}(Q, K, V)$ denotes a gated cross-attention layer, in which the query embeddings attend to the key and value embeddings to produce cross-modal interaction features, and a learnable gating function with sigmoid activation is applied to control how much of these features are preserved in the fused embeddings, $\mathbf{e}_{\text{task}} = \mathcal{T}(p_{\text{task}})$, where $\mathbf{e}_{\text{task}} \in \mathbb{R}^{N_{\text{task}} \times d}$ denotes the task embeddings, N_{task} denotes the length of the tokenized task description prompt p_{task} . This gating mechanism is introduced to enable more flexible and effective multi-modal fusion.

By replacing Eq. (5) with Eq. (10), we leverage the cross-attention mechanism to enable trajectory embeddings to adaptively extract relevant information from text embeddings, thereby naturally aligning the two modalities in the embedding space. Notably, this fusion process preserves the original trajectory length without introducing additional text tokens. As a result, the proposed module achieves

effective multi-modal fusion and embedding alignment while avoiding the computational overhead caused by lengthy inputs, effectively addressing the second limitation of previous methods.

3.4 Constraint-Gated LoRA

Previous methods typically rely on full fine-tuning to adapt the LLM backbone for auto-bidding, since the simple fusion strategy cannot effectively bridge the embedding space discrepancy between trajectory and text modalities. This paradigm incurs substantial computational overhead and limits the transferability of the model to new bidding scenarios and constraint settings. In Section 3.3, the proposed text alignment module achieves effective cross-modal alignment through semantic interaction, thereby enabling parameter-efficient fine-tuning for model adaptation.

To do this, we propose the constraint-gated LoRA module, which performs parameter-efficient fine-tuning on the frozen LLM backbone through a mixture-of-experts (MoE) LoRA. LoRA [19] introduces trainable low-rank adapters into the original weight matrices and has demonstrated strong performance in various transfer learning tasks [8, 35, 39, 53], achieving performance comparable to full fine-tuning while updating a small number of parameters. Therefore, LoRA provides an effective solution for efficient adaptation in auto-bidding scenarios.

However, advertisers adopt different bidding strategies under diverse optimization objectives and constraints, such as maximizing conversions under CPA constraints, making it difficult for a single LoRA adapter to capture heterogeneous bidding behaviors. Inspired by MoE methods [35, 55], we introduce multiple strategy-specific LoRA experts and employ condition-aware routing to dynamically select appropriate experts according to advertiser constraints. To do this, we first introduce a condition constraint prompt p_{con} to describe advertiser objectives and constraints, and employ a gated cross-attention mechanism to enable the trajectory to extract relevant information from the condition constraint for expert selection. The formulation can be written as follows:

$$C^l = \text{Gated_attn}(Q = H^l, K = e_{\text{con}}, V = e_{\text{con}}), \quad C^l \in \mathbb{R}^{L \times d}, \quad (11)$$

where

$$C^l = (c_{0,k=R}^l, c_{0,k=S}^l, c_{0,k=A}^l, \dots, c_{T-1,k=A}^l), \quad (12)$$

where H^l denotes the hidden states after the causal self-attention, residual connection, and layer normalization at the l -th transformer layer, C^l denotes the constraint information extracted by the trajectory at the l -th layer, and $e_{\text{con}} = \mathcal{T}(p_{\text{con}})$, where $e_{\text{con}} \in \mathbb{R}^{N_{\text{con}} \times d}$ denotes the condition embeddings, N_{con} is the length of the tokenized condition constraint prompt. Then, we employ a multi-layer perceptron MLP($\cdot$) to aggregate the information extracted by different semantic tokens within the same timestep, which serves as the input to the MoE gating mechanism. The formulation is as follows:

$$y_t^l = \text{MLP}([c_{t,k=R}^l, c_{t,k=S}^l, c_{t,k=A}^l]), \quad y_t^l \in \mathbb{R}^d, \quad (13)$$

where y_t^l denotes the condition gating signal at timestep t in the l -th layer. Since y_t^l encodes the constraint-aware information of the current token, we use it as the input to the MoE-LoRA gated network to dynamically select the appropriate LoRA expert for each trajectory token, thereby enabling adaptive strategy selection.

The formulation is as follows:

$$\begin{cases} g_t^l = \text{Softmax}(\text{Router}^l(y_t^l)) \\ m_t^l = \arg \max_m g_t^l, \end{cases} \quad (14)$$

where $\text{Router}^l(\cdot)$ denotes the gating network at l -th layer, $g_t^l \in \mathbb{R}^M$ denotes the routing weight over experts at l -th layer and timestep t , M denotes the number of experts, and m_t^l denotes the index of the selected expert at l -th layer and timestep t . After selecting the strategy expert, we adopt the standard LoRA scheme [19] to perform low-rank adaptation on the backbone network. Specifically, the output of the linear layer is computed as the sum of the original backbone output and the increment produced by the low-rank adapter. The formulation is given as follows:

$$\tilde{H}_t^l = H_t^l W^\top + \frac{\alpha}{r} H_t^l A_{m_t^l}^\top B_{m_t^l}^\top, \quad (15)$$

where $\tilde{H}_t^l \in \mathbb{R}^{3 \times d}$ denotes the updated hidden state after the transformation, $H_t^l \in \mathbb{R}^{3 \times d}$ denotes the hidden state at timestep t of H^l in the l -th layer, W denotes the base weight matrix of the linear transformation, α denotes the LoRA scaling factor, r denotes the rank of the low-rank adaptation, where $r \ll d$ to ensure parameter-efficient adaptation, $A_{m_t^l}$ denotes the low-rank down-projection matrix, and $B_{m_t^l}$ denotes the low-rank up-projection matrix.

Through Eq. (15), domain adaptation can be achieved by training only a small number of parameters while keeping the backbone frozen. Furthermore, the constraint-gated LoRA module models the interaction between trajectory information and constraint condition to generate timestep-wise gating signals, and leverages a MoE LoRA to enable efficient fine-tuning and flexible policy selection, thereby addressing the third limitation of previous methods.

Overall, the three proposed modules jointly address the three aforementioned key challenges.

4 Experiments

4.1 Experimental Details

Datasets. To evaluate the effectiveness of our method, we use AuctionNet [47], a large-scale advertising bidding dataset constructed by Alibaba. The dataset contains two variants, namely a dense version and a sparse version, referred to as AuctionNet-dense and AuctionNet-sparse. Each dataset includes data from 48 advertisers, with more than 500,000 impression opportunities in each period. More details about the dataset are provided in Appendix A.2.

Comparison methods. The comparison methods include four offline RL methods (*i.e.*, USCB [18], BCQ [12], CQL [25], IQL [24]), one planning-based generative bidding method (*i.e.*, DiffBid [16]), and three sequence modeling-based methods (DT [4], CDT [33], DT-score), where DT-score further fine-tunes DT using Q-values [30]. We also compare with five LLM-based methods (*i.e.*, Prompting, SFT, GRPO [46], LLM-DT, Prompt-LLM-DT) for decision-making. Specifically, Prompting, SFT, and GRPO operate within the language modality, while LLM-DT and Prompt-LLM-DT process numerical sequences to generate numerical actions, with Prompt-LLM-DT further incorporating a language task description [29].

Implementation details. Except for the ablation studies, our method is built upon the pretrained Qwen2.5-Instruct-0.5B model [3]. During training, the backbone parameters and the text embedder

Table 1: Score comparison between our proposed method and previous baselines on the AuctionNet benchmark under different budget scales. Bold indicates the best performance, and underlined values denote the second-best performance.

Dataset	Budget	USCB	BCQ	CQL	IQL	DiffBid	DT	CDT	DT-score	SAGE	Improve
AuctionNet-dense	50%	86	190	113	164	54	<u>191</u>	174	178	204	6.81%
	75%	135	259	139	232	100	265	242	<u>268</u>	274	2.24%
	100%	157	321	171	281	152	329	326	<u>334</u>	349	4.49%
	125%	220	379	201	355	193	<u>396</u>	378	395	403	1.77%
	150%	281	429	238	401	234	<u>450</u>	433	441	462	2.67%
AuctionNet-sparse	50%	11.5	17.7	12.8	16.5	9.9	14.8	11.2	<u>17.7</u>	19.6	10.73%
	75%	14.9	24.6	16.7	22.1	15.4	22.9	18.0	<u>25.9</u>	29.7	14.67%
	100%	17.5	31.1	22.2	30.0	19.5	29.6	31.2	<u>33.2</u>	35.8	7.83%
	125%	26.7	34.2	28.6	37.1	25.3	34.3	31.7	<u>39.6</u>	40.8	3.03%
	150%	31.3	37.9	35.8	43.1	30.8	44.5	39.1	<u>44.7</u>	46.5	4.03%

Table 2: Score and conversions comparison between our proposed method and LLM-based multi-modal methods on the AuctionNet benchmark under different budget scales. Bold indicates the best performance, and underlined values denote the second-best performance.

Dataset	Metric	Prompting	SFT	GRPO	LLM-DT	Prompt-LLM-DT	SAGE	Improve
AuctionNet-dense	Conversions ($\uparrow$)	289	297	327	<u>366</u>	355	371	1.37%
	Score ($\uparrow$)	286	286	292	<u>328</u>	322	349	6.40%
AuctionNet-sparse	Conversions ($\uparrow$)	24.2	27.0	29.9	<u>36.0</u>	35.8	38.3	6.39%
	Score ($\uparrow$)	23.4	26.9	29.7	<u>32.5</u>	30.3	35.8	10.15%

remain frozen. The batch size is 128, the learning rate is 1e-5, and AdamW [34] is used as the optimizer. The length of the trajectory is 10, and the maximum number of training steps is 400,000. More implementation details are provided in Appendix A.3.

Evaluation metrics. The evaluation metrics mainly include the following two measures:

- Conversions: the total number of conversions within one advertising period without constraints, defined as $\sum_i o_i v_i$.
- Score: the value under CPA constraints. A penalty defined as $\min\left\{\left(\frac{1}{\text{ratio}}\right)^2, 1\right\}$ is applied, where ratio is the ratio between the real CPA and the target CPA. The score is defined as $\sum_i o_i v_i \times \text{penalty}$.

4.2 Main Results

We report the experimental results of all methods on the AuctionNet benchmark, including its two variants, AuctionNet-dense and AuctionNet-sparse. Table 1 compares our proposed method with traditional auto-bidding methods, while Table 2 compares our method with several LLM-based multi-modal methods.

Comparison to traditional baselines. Our method achieves the best results, followed by DT-score, DT, CDT, BCQ, IQL, CQL, USCB, and DiffBid, in terms of different budget scales on all variants. For instance, under the 100% budget setting, our method improves by 4.49% and 7.83%, respectively, compared to the best comparison method (*i.e.*, DT-score) on the AuctionNet-dense and AuctionNet-sparse variant, and by 129.60% and 83.59% compared to the worst comparison method (*i.e.*, DiffBid) in terms of score. This performance improvement may stem from the introduction of multi-modal information in our method, which enables the model to leverage the rich pretrained prior knowledge of LLMs. Moreover, by designing prompts to explicitly express and reinforce constraints,

the model is guided to learn decision strategies that better satisfy these constraints, thereby achieving improved performance.

Comparison to LLM-based methods. Our method achieves the best results, followed by LLM-DT, Prompt-LLM-DT, GRPO, SFT, Prompting, in terms of different budget scales on all variants. For instance, under the 100% budget setting, our method improves by 6.40% and 10.15%, respectively, compared to the best comparison method (*i.e.*, LLM-DT) on the AuctionNet-dense and AuctionNet-sparse variants, and by 22.03% and 52.99% compared to the worst comparison method (*i.e.*, Prompting) in terms of score. One possible reason is that our method introduces distinctive semantic embeddings for the bidding trajectory, which enhances the model’s understanding of different types of tokens. Notably, our method achieves larger improvements on the sparse variants, which may be attributed to the proposed text alignment module that leverages a cross-attention mechanism to enable more effective fusion and alignment between the trajectory modality and the text modality. As a result, our method achieves better performance compared with previous LLM-based methods. Moreover, it is also worth noting that LLM-based methods generally achieve strong performance, which further indicates that incorporating LLMs and utilizing sufficient rich pretrained prior knowledge can effectively improve the bidding decision capability in auto-bidding tasks.

Table 3: Ablation studies on position augmentation and text alignment.

Method	Score $\uparrow$
w/o position augmentation	243
w/o text alignment	324
w/o position augmentation and text alignment	212
SAGE	349

4.3 Ablation Studies

Effectiveness of position augmentation and text alignment. The proposed method involves three key components, *i.e.*, the position augmentation, the text alignment, and the constraint-gated LoRA. Since the ablation of the constraint-gated LoRA module involves modifications to both the fine-tuning strategy and the text prompt, we defer the analysis of its effectiveness to a later section. In this section, we first evaluate the effectiveness of the position augmentation and text alignment modules. To this end, we report the score performance on the AuctionNet benchmark after ablating different components in Table 3. The experimental results demonstrate that each component of our framework is effective. Specifically, compared with the full model, removing the position augmentation module leads to a decrease of 30.37% in score, indicating that introducing distinctive semantic embeddings for trajectory information helps strengthen the distinction between different types of tokens and thus improves the model’s discriminative capability. When the text alignment module is removed, the score drops by 7.16%, suggesting that leveraging cross-attention to enable interaction between the trajectory modality and the text modality can effectively facilitate cross-modal fusion and alignment, thereby improving the model’s understanding of the auto-bidding task. When both modules are removed, the performance further decreases by 39.26%, which indicates that the proposed components work collaboratively and jointly contribute to achieving the best performance.

Effectiveness of different positional embeddings. Since the proposed position augmentation module consists of both semantic and temporal embeddings, we further analyze the contribution of each component and compare them with the conventional method that assigns learnable positional embeddings to each timestep. As shown in Table 4, we report the score performance and parameter counts on the AuctionNet benchmark when using temporal embeddings, semantic embeddings, and traditional positional embeddings, respectively. The results show that introducing semantic

Table 4: Experiments on the impact of different positional embeddings, dim denotes the hidden dimension, dim’ denotes the dimension of traditional position embeddings, and timestep denotes the total timestep in the trajectory.

Method	Score ↑	Parameter counts ↓
w/o semantic embeddings	249	0
w/o temporal embeddings	315	$3 \times \text{dim}$
traditional position embeddings	250	$\text{timestep} \times \text{dim}' + (\text{dim} + \text{dim}') \times \text{dim}$
SAGE	349	$3 \times \text{dim}$

embedding significantly improves performance, demonstrating the effectiveness of the proposed design. Moreover, compared with traditional learnable positional embeddings, the sinusoidal temporal embeddings achieve comparable performance without introducing additional parameters and remains effective when the number of timestep changes, highlighting its efficiency and scalability.

Effectiveness of constraint-gated LoRA. To evaluate the effectiveness of the proposed constraint-gated LoRA module, we conduct ablation studies from two perspectives: prompt design and fine-tuning strategy. Since directly removing the module simultaneously affects both the utilization of constraint information and the parameter adaptation mechanism, we design more fine-grained comparisons. Specifically, after removing LoRA, we consider two

training strategies: full fine-tuning and frozen parameters. Meanwhile, for prompt design, we compare removing the constraint information with concatenating it to the task description. The results are reported in Table 5. The results show that preserving constraint information consistently improves performance, indicating that it provides important guidance for bidding decisions. In addition, freezing the backbone parameters performs better than full fine-tuning. A possible reason is that full fine-tuning a large language model may disrupt its rich pretrained prior knowledge, whereas keeping the backbone frozen helps retain its general capabilities and leads to more stable decision-making. Overall, the proposed constraint-gated LoRA module effectively leverages constraint information through a gating mechanism and performs efficient parameter adaptation via LoRA, which together contribute to improved performance. In addition to the superior performance, compared with full fine-tuning, our method uses less than 10% of the trainable parameters while achieving effective LLM adaptation. These results demonstrate both the efficiency and effectiveness of the proposed method.

Scalability across different backbone models. To evaluate

Table 5: Ablation studies on the effectiveness of the constraint-gated LoRA module.

LoRA	Condition	Full FT	Score↑	Trainable Params (M)↓
✗	✗	✗	236	22.7
✗	concat	✗	334	22.7
✗	✗	✓	144	380.6
✗	concat	✓	173	380.6
✓	gated	✗	349	31.6

Table 6: Experiments on AuctionNet benchmark with different backbone architectures.

Method	AuctionNet-dense		AuctionNet-sparse	
	Score↑	Conversions↑	Score↑	Conversions↑
Qwen2.5-Instruct-0.5B	349	371	35.8	38.3
Qwen2.5-Instruct-1.5B	347	370	33.4	35.6
Qwen2.5-Instruct-3B	344	369	35.0	36.5

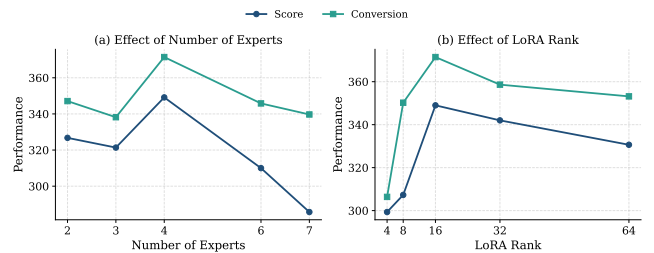


Figure 2: Experiments on AuctionNet benchmark with different hyperparameter settings.

the scalability of our method, we conduct experiments on different backbone models. The results reported in Table 6 indicate that our method consistently achieves strong performance across all architectures, demonstrating its effectiveness and scalability. We also observe a slight performance drop on larger models. One possible

reason is that adapting larger models becomes more challenging under limited training data, making it harder for the parameters to converge effectively. In addition, prior studies suggest that more precise representations do not necessarily lead to better task performance [59]; instead, the alignment between model representations and the target task also plays an important role.

Hyperparameter sensitivity analysis. To analyze the impact of the number of MoE experts M and the LoRA rank r on model performance, we report the results under different configurations in Figure 2. The results show that our method achieves strong and stable performance across a certain range of settings by $M \in \{2, 3, 4\}$ and $r \in \{16, 32, 64\}$. Notably, when the number of experts becomes large (e.g., $M = 7$), the performance drops noticeably. This may be because increasing the number of experts makes it more difficult for the routing network to learn stable assignment patterns, which can lead to imbalanced or inaccurate expert selection and thus degrade the overall performance. In addition, performance also decreases when the LoRA rank is too small (e.g., $r = 4$), since a low rank restricts the expressive capacity of parameter updates, making it harder for the model to adapt to the task.

MoE load balancing analysis. We further analyze the MoE load balancing in Appendix A.4.

5 Conclusion

In this paper, we propose **SAGE**, a new LLM-guided framework for auto-bidding. Specifically, we first design a position augmentation module to enhance trajectory modeling with temporal-semantic positional embeddings. We then introduce a text alignment module to align trajectory and text information through cross-attention. Finally, we propose a constraint-gated LoRA module that uses constraints as gating signals to select experts while enabling parameter-efficient fine-tuning of pretrained LLMs. Experimental results demonstrate the effectiveness and efficiency of our method.

Acknowledgments

This work was supported by Alibaba Group through Alibaba Research Intern Program.

References

- [1] Anurag Ajay, Yilun Du, Abhi Gupta, Joshua B Tenenbaum, Tommi S Jaakkola, and Pulkit Agrawal. 2023. Is Conditional Generative Modeling all you need for Decision Making?. In *ICLR*. 1–24.
- [2] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. 2022. Flamingo: a visual language model for few-shot learning. In *NeurIPS*. 23716–23736.
- [3] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. *arXiv* (2023), 1–59.
- [4] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Misha Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. 2021. Decision transformer: Reinforcement learning via sequence modeling. In *NeurIPS*. 15084–15097.
- [5] Ye Chen, Pavel Berkhin, Bo Anderson, and Nikhil R. Devanur. 2011. Real-time bidding algorithms for performance-based display ad allocation. In *ICML*. 1–23.
- [6] Miduo Cui, Haochen Wang, Shangqin Mao, Xun Yang, Qianlong Xie, Xingxing Wang, Xuri Ge, Ying Zhou, and Zhiwei Xu. 2026. DRIVE: Distributional and Retrieval-Augmented Bidding with Value Evaluation. In *ICML*. 1–23.
- [7] Yaodong Cui, Shucheng Huang, Jiaming Zhong, Zhenan Liu, Yutong Wang, Chen Sun, Bai Li, Xiao Wang, and Amir Khajepour. 2023. Drivellm: Charting the path toward full autonomous driving with large language models. *TIJ* (2023), 1450–1464.
- [8] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms. In *NeurIPS*. 10088–10115.
- [9] Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, Bill Yuchen Lin, Sean Welleck, Peter West, Chandra Bhagavatula, Ronan Le Bras, et al. 2023. Faith and fate: Limits of transformers on compositionality. In *NeurIPS*. 70293–70332.
- [10] David S Evans. 2009. The online advertising industry: Economics, evolution, and privacy. *JEP* (2009), 37–60.
- [11] Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. 2025. Group-in-group policy optimization for llm agent training. In *NeurIPS*. 46375–46408.
- [12] Scott Fujimoto, David Meger, and Doina Precup. 2019. Off-policy deep reinforcement learning without exploration. In *ICML*. 2052–2062.
- [13] Jingtong Gao, Yewen Li, Shuai Mao, Peng Jiang, Nan Jiang, Yejing Wang, Qingpeng Cai, Fei Pan, Peng Jiang, Kun Gai, et al. 2025. Generative auto-bidding with value-guided explorations. In *SIGIR*. 244–254.
- [14] Yulong Gao, Wan Jiang, Mingzhe Cao, Xuepu Wang, Zeyu Pan, Haonan Yang, Ye Liu, and Xin Yang. 2025. SEGB: Self-Evolved Generative Bidding with Local Autoregressive Diffusion. *arXiv* (2025), 1–11.
- [15] Mozahdeh Gheini, Xiang Ren, and Jonathan May. 2021. Cross-attention is all you need: Adapting pretrained transformers for machine translation. In *EMNLP*. 1754–1765.
- [16] Jiayan Guo, Yusen Huo, Zhilin Zhang, Tianyu Wang, Chuan Yu, Jian Xu, Bo Zheng, and Yan Zhang. 2024. Generative auto-bidding via conditional diffusion modeling. In *KDD*. 5038–5049.
- [17] Qianyu Hao, Yiwen Song, Qingmin Liao, Jian Yuan, and Yong Li. 2026. Llm-explorer: A plug-in reinforcement learning policy exploration enhancement driven by large language models. In *NeurIPS*. 93586–93627.
- [18] Yue He, Xiujun Chen, Di Wu, Junwei Pan, Qing Tan, Chuan Yu, Jian Xu, and Xiaoqiang Zhu. 2021. A unified solution to constrained bidding in online display advertising. In *KDD*. 2993–3001.
- [19] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *ICLR*. 1–26.
- [20] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. 2025. $\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization. *arXiv* (2025), 1–19.
- [21] Michael Janner, Yilun Du, Joshua Tenenbaum, and Sergey Levine. 2022. Planning with Diffusion for Flexible Behavior Synthesis. In *ICML*. 9902–9915.
- [22] Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, et al. 2024. Time-llm: Time series forecasting by reprogramming large language models. In *ICLR*. 23857–23880.
- [23] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P Foster, Pannag R Sanketi, Quan Vuong, et al. 2025. OpenVLA: An Open-Source Vision-Language-Action Model. In *CoRL*. 1–37.
- [24] Ilya Kostrikov, Ashvin Nair, and Sergey Levine. 2022. Offline Reinforcement Learning with Implicit Q-Learning. In *ICLR*. 1–13.
- [25] Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. 2020. Conservative q-learning for offline reinforcement learning. In *NeurIPS*. 1179–1191.
- [26] Yu Lei, Jiayang Zhao, Yilei Zhao, Zhaoqi Zhang, Linyou Cai, Qianlong Xie, and Xingxing Wang. 2026. Generative Large-Scale Pre-trained Models for Automated Ad Bidding Optimization. In *KDD*. 2266–2277.
- [27] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. 2023. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *ICML*. 19730–19742.
- [28] Yewen Li, Jingtong Gao, Nan Jiang, Shuai Mao, Ruyi An, Fei Pan, Xiangyu Zhao, Bo An, Qingpeng Cai, and Peng Jiang. 2025. Generative Auto-Bidding in Large-Scale Competitive Auctions via Diffusion Completer-Aligner. *arXiv* (2025).
- [29] Yewen Li, Zhiyi Lyu, Peng Jiang, Qingpeng Cai, Fei Pan, Bo An, and Peng Jiang. 2026. LBM: Hierarchical Large Auto-Bidding Model via Reasoning and Acting. In *WWW*. 75–86.
- [30] Yewen Li, Shuai Mao, Jingtong Gao, Nan Jiang, Yunjian Xu, Qingpeng Cai, Fei Pan, Peng Jiang, and Bo An. 2025. Gas: Generative auto-bidding with post-training search. In *WWW*. 315–324.
- [31] Yongkang Li, Kaixin Xiong, Xiangyu Guo, Fang Li, Sixu Yan, Gangwei Xu, Lijun Zhou, Long Chen, Haiyang Sun, Bing Wang, et al. 2025. Recogdrive: A reinforced cognitive framework for end-to-end autonomous driving. *arXiv* (2025), 1–37.
- [32] Bingyan Liu, Chengyu Wang, Tingfeng Cao, Kui Jia, and Jun Huang. 2024. Towards understanding cross and self-attention in stable diffusion for text-guided image editing. In *CVPR*. 7817–7826.
- [33] Zuxin Liu, Zijian Guo, Yihang Yao, Zhepeng Cen, Wenhao Yu, Tingnan Zhang, and Ding Zhao. 2023. Constrained decision transformer for offline safe reinforcement learning. In *ICML*. 21611–21630.
- [34] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. In *ICLR*. 1–18.
- [35] Tongxu Luo, Jiahe Lei, Fangyu Lei, Weihao Liu, Shizhu He, Jun Zhao, and Kang Liu. 2024. Moelora: Contrastive learning guided mixture of experts on parameter-efficient fine-tuning for large language models. *arXiv* (2024), 1–12.

- [36] Xiaowei Lv, Zhilin Zhang, Yijun Li, Yusen Huo, Siyuan Ju, Xuyan Li, Chunxiang Hong, Tianyu Wang, Yongcai Wang, Peng Sun, et al. 2026. DecisionLLM: Large Language Models for Long Sequence Decision Exploration. *arXiv* (2026), 1–14.
- [37] Zhiyu Mou, Yusen Huo, Rongquan Bai, Mingzhou Xie, Chuan Yu, Jian Xu, and Bo Zheng. 2022. Sustainable online reinforcement learning for auto-bidding. In *NeurIPS*. 2651–2663.
- [38] S. Muthukrishnan. 2009. Ad Exchanges: Research Issues. In *WINE*. 1–12.
- [39] Omid Nejatimanzari, Hojat Asgariandehkordi, Taha Koleilat, Yiming Xiao, and Hassan Rivaz. 2026. Sparse Spectral LoRA: Routed Experts for Medical VLMs. In *CVPR*. 35351–35362.
- [40] Jing-Cheng Pang, Si-Hang Yang, Kaiyuan Li, Xiong-Hui Chen, Nan Tang, and Yang Yu. 2024. Kalm: Knowledgeable agents by offline reinforcement learning from large language model rollouts. In *NeurIPS*. 126620–126652.
- [41] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. 2025. Fast: Efficient action tokenization for vision-language-action models. *arXiv* (2025), 1–19.
- [42] Akshara Prabhakar, Zuxin Liu, Ming Zhu, Jianguo Zhang, Tulika Manoj Awalganekar, Shiyu Wang, Zhiwei Liu, Haolin Chen, Thai Hoang, Juan Carlos Niebles, et al. 2025. Apigen-mt: Agentic pipeline for multi-turn data generation via simulated agent-human interplay. In *NeurIPS*. 1–18.
- [43] Kan Ren, Weinan Zhang, Ke Chang, Yifei Rong, Yong Yu, and Jun Wang. 2017. Bidding machine: Learning to bid for directly optimizing profits in display advertising. *TKDE* (2017), 645–659.
- [44] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2022. High-resolution image synthesis with latent diffusion models. In *CVPR*. 10684–10695.
- [45] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv* (2017), 1–12.
- [46] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv* (2024), 1–30.
- [47] Kefan Su, Yusen Huo, Zhilin Zhang, Shuai Dou, Chuan Yu, Jian Xu, Zongqing Lu, and Bo Zheng. 2024. Auctionnet: A novel benchmark for decision-making in large-scale games. In *NeurIPS*. 94428–94452.
- [48] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *NeurIPS*. 6000–6010.
- [49] Xu Wan, Wenyue Xu, Chao Yang, and Mingyang Sun. 2025. Think Twice, Act Once: A Co-Evolution Framework of LLM and RL for Large-Scale Decision Making. In *ICML*. 62102–62120.
- [50] Jun Wang and Shuai Yuan. 2015. Real-time bidding: A new frontier of computational advertising research. In *WSDM*. 415–416.
- [51] Chao Wen, Miao Xu, Zhilin Zhang, Zhenzhe Zheng, Yuhui Wang, Xiangyu Liu, Yu Rong, Dong Xie, Xiaoyang Tan, Chuan Yu, et al. 2022. A cooperative-competitive multi-agent framework for auto-bidding in online advertising. In *WSDM*. 1129–1139.
- [52] Xun Yang, Yasong Li, Hao Wang, Di Wu, Qing Tan, Jian Xu, and Kun Gai. 2019. Bid optimization by multivariable control in display advertising. In *KDD*. 1966–1974.
- [53] Yaming Yang, Dilxat Muhtar, Yelong Shen, Yuefeng Zhan, Jianfeng Liu, Yujing Wang, Hao Sun, Weiwei Deng, Feng Sun, Qi Zhang, et al. 2025. Mtl-lora: Low-rank adaptation for multi-task learning. In *AAAI*. 22010–22018.
- [54] Hao Yu, Michael Neely, and Xiaohan Wei. 2017. Online Convex Optimization with Stochastic Constraints. In *NeurIPS*. 1–11.
- [55] Dacao Zhang, Kun Zhang, Shimao Chu, Le Wu, Xin Li, and Si Wei. 2025. More: A mixture of low-rank experts for adaptive multi-task learning. In *ACL*. 1311–1324.
- [56] Mingming Zhang, Feiqing Zhuang, Na Li, Shengjie Sun, Xiaowei Chen, Junxiong Zhu, Fei Xiao, Keping Yang, Lixin Zou, and Chenliang Li. 2026. Generative Auto-Bidding with Unified Modeling and Exploration. *arXiv* (2026), 1–11.
- [57] Weinan Zhang, Yifei Rong, Jun Wang, Tianchi Zhu, and Xiaofan Wang. 2016. Feedback control of real-time display advertising. In *WSDM*. 407–416.
- [58] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. 2024. Webarena: A realistic web environment for building autonomous agents. In *ICLR*. 15585–15606.
- [59] Guanyu Zhu, Jining Luan, Hanwen Du, Xinyu Fang, Sibo Xu, Ersheng Ni, Hongji Li, Jincheng Fang, Ronghao Chen, Huacan Wang, et al. 2026. On the Role of Language Representations in Auto-Bidding: Findings and Implications. *arXiv* (2026), 1–19.

A APPENDIX

A.1 Related Work

Auto-bidding methods. With the rapid growth of e-commerce platforms, auto-bidding for online advertising has become an important research topic. Early studies mainly relied on control-based methods [5, 52, 54, 57], such as PID [5] and OnlineLP [54], which adjusted bidding strategies through manually designed optimization or control rules. However, as auction environments become increasingly dynamic and the scale of advertising traffic continues to grow, these methods face significant challenges in scalability and adaptability. Benefiting from the rapid development of RL and strong baseline models, recent auto-bidding methods can generally be categorized into four groups: traditional offline RL methods, sequence modeling-based methods, planning-based generative bidding methods, and LLM-based multi-modal methods. Traditional offline RL methods learn bidding policies by optimizing long-term rewards through value estimation and policy optimization [24, 25, 37, 51]. For example, SORL [37] combines offline pretraining with online RL to improve bidding stability, while MAAB [51] models auto-bidding as a cooperative-competitive multi-agent optimization problem. Sequence modeling-based methods reformulate auto-bidding as a sequential prediction task and directly generate bidding actions from historical trajectories [4, 13, 26, 56]. For instance, GAS [30] enhances generative bidding through post-training search, while DRIVE [6] leverages distributional modeling and retrieval augmentation to improve decision quality. Planning-based generative bidding methods further incorporate future planning mechanisms into generative policies to guide current bidding decisions [14, 16, 21, 28]. For example, DiffBid [16] models bidding behaviors through conditional diffusion processes, while SEGB [14] employs self-evolved autoregressive diffusion for long-horizon bidding planning. Recently, LLM-based multi-modal methods have attracted increasing attention due to their strong reasoning and language understanding capabilities. These methods leverage text information to model advertiser objectives and constraints while benefiting from the long-context reasoning ability of LLMs [29, 36, 59]. For example, LBM [29] adopts a hierarchical reasoning-and-acting framework for bidding decision generation, while SemBid [59] investigates the role of language representations in improving auto-bidding performance.

LLM for sequential decision-making. Recently, LLMs have been widely applied to sequential decision-making tasks such as web navigation [58], robotics [20, 23, 41], and autonomous driving [7, 31] due to their strong reasoning and semantic modeling capabilities. However, since LLM representations are inherently derived from discrete text modalities, they struggle to effectively model the continuous, high-dimensional, and dynamic state spaces and behavioral trajectories in sequential decision-making environments [9]. To enhance the capabilities of LLMs in sequential decision-making tasks, recent studies have adopted supervised fine-tuning methods that train models on manually annotated high-quality data, thereby improving their adaptability to complex environments. For example, APiGen-MT [42] improves the adaptability of LLMs in multi-turn interaction and sequential decision-making tasks by constructing a simulated human-agent interaction framework to automatically generate high-quality multi-turn trajectory data for supervised

fine-tuning. In addition, another line of research enhances the sequential decision-making capabilities of LLMs through RL, where PPO [45] and its variants are widely adopted for policy optimization and long-term reward modeling [11, 46]. Some studies further incorporate modular designs, such as exploration [17] and data augmentation [40, 49], to improve decision performance.

A.2 Dataset Details

Table 7: Parameters of AuctionNet-dense and AuctionNet-Sparse.

Parameters	AuctionNet-dense	AuctionNet-sparse
Trajectories	479,376	479,376
Delivery periods	9,987	9,987
timestep in a trajectory	48	48
RTG dimension	1	1
State dimension	16	16
Action dimension	1	1
Action range	[0, 493]	[0, 589]
Impression's value range	[0, 1]	[0, 1]
CPA range	[6, 12]	[60, 130]
Total conversion range	[0, 1512]	[0, 57]

In our experiments, we adopt AuctionNet [47], a large-scale public advertising bidding benchmark proposed and constructed by Alibaba. Specifically, the dataset provides two variants with different conversion sparsity levels, referred to in this paper as AuctionNet-dense and AuctionNet-sparse. Each variant contains over 500000 impression opportunities from 48 advertisers across 9987 delivery periods. Detailed dataset statistics are shown in Table 7. In addition, the state at each timestep in a trajectory is constructed by aggregating historical statistics and current budget-related information. Specifically, each dimension represents the following information:

- time_left:** The number of remaining decision intervals in the current advertising delivery period.
- budget_left:** The remaining budget available for allocation in the current delivery period.
- historical_bid_mean:** The average bid submitted by the advertiser over all previous timesteps.
- last_three_bid_mean:** The moving average bid over the last three timesteps.
- historical_LeastWinningCost_mean:** The average market clearing price over all previous timesteps, *i.e.*, the minimum cost required to win an auction.
- last_three_LeastWinningCost_mean:** The average market clearing price over the last three timesteps.
- historical_pValues_mean:** The average predicted conversion probability over all previous timesteps.
- last_three_pValues_mean:** The average predicted conversion probability over the last three timesteps.
- current_pValues_mean:** The average predicted conversion probability of impression opportunities arriving at the current timestep.
- historical_conversion_mean:** The average number of conversions achieved over all previous timesteps.

last_three_conversion_mean: The average number of conversions over the last three timesteps.

historical_xi_mean: The historical average winning rate, defined as the ratio between the number of won auctions and the total number of participated auctions.

last_three_xi_mean: The average winning rate over the last three timesteps.

current_pv_num: The total number of available impression opportunities at the current timestep.

last_three_pv_num_total: The cumulative number of impression opportunities over the last three timesteps.

historical_pv_num_total: The cumulative number of impression opportunities over all previous timesteps.

Table 8: Detailed hyperparameter settings for SAGE.

Hyperparameter	Value
LLM backbone	Qwen2.5-Instruct-0.5B
Batch size	128
Learning rate	1e-5
Optimizer	AdamW
Weight decay	1e-4
Grad clip	0.25
Length of trajectory	10
Episode length	48
Total training steps	400000
Number of cross-attention head	8
Number of experts	4
LoRA rank	16
Scaling factor	16

A.3 Implementation Details

Table 8 presents the detailed hyperparameter configurations of the proposed SAGE method. Notably, the hyperparameter settings remain largely consistent across different benchmark variants, which further demonstrates the robustness of our method.

A.4 MoE Load Balancing Analysis

In our constraint-gated LoRA module, we do not explicitly introduce an auxiliary load-balancing loss. This design is motivated by the fact that each expert is intended to capture a distinct bidding strategy, and the strategy patterns are relatively separable under different constraint conditions. As a result, different tokens can be naturally routed to their corresponding experts based on the learned gating mechanism.

Nevertheless, to verify whether the routing process remains balanced in practice, we further report the expert utilization statistics during inference when the number of experts is set to 4. As shown in Table 9, the routing counts of the four experts are 445,752, 311,151, 435,567, and 324,570, respectively, corresponding to relatively balanced selection ratios of 29.38%, 20.51%, 28.71%, and 21.39%. These results indicate that the gating network does not collapse to a single expert, suggesting that the proposed constraint-gated LoRA module can effectively allocate different constraints to different experts without requiring an additional load balancing objective.

Table 9: Expert utilization statistics during inference when the number of experts is set to 4.

Expert	Selection Count	Selection Ratio
Expert 1	445,752	29.38%
Expert 2	311,151	20.51%
Expert 3	435,567	28.71%
Expert 4	324,570	21.39%

A.5 Text Prompt

The task description prompt template and the condition constraint prompt template used in our experiments are provided as follows:

Task Description Prompt

You are an expert digital advertising strategist, embodied as an advanced auto-bidding algorithm.

Input Format

You receive a fixed-length trajectory segment containing $K = 10$ timesteps.

Each timestep t is represented by three tokens in the following order:

$$(RTG_t, S_t, A_t)$$

where:

- RTG_t : return-to-go at timestep t
- S_t : observed state at timestep t
- A_t : action taken at timestep t

Decision Objective

At the current timestep t , predict the bidding multiplier α for that timestep.

Condition Constraint Prompt

Primary Goal

Your primary goal is to maximize the total number of conversions for an ad campaign over a full day (the horizon), while strictly adhering to the budget and CPA constraints.

CPA Definition (Simple)

CPA is defined as:

$$CPA = \frac{\text{total_spend}}{\max(\text{total_conversions}, 1)}$$

For decision-making at the current timestep, approximate the realized CPA using ONLY information observed so far in the trajectory (do NOT use future information).

CPA Reliability Principle

Early in the campaign, when only a very small amount of budget has been spent or when conversions are very sparse, the realized CPA estimate is highly unreliable and should not dominate the bidding decision.

In such early stages:

- Do not overreact to temporarily high CPA values caused by very small spend or zero conversions.
- Maintain a reasonable bidding level to gather sufficient impressions and conversions so that the CPA estimate becomes statistically meaningful.

As more budget is spent and more conversions are observed, gradually place more weight on the CPA constraint and adjust bids accordingly.

Guiding Principles

You must follow these two core principles to achieve your goal:

1. Dynamic CPA Management

- If the realized CPA is reliably higher than the target after sufficient spending, decrease the bidding multiplier. However, when spending is still very small or conversions are sparse, avoid aggressively lowering bids based on unstable CPA estimates.

2. Smooth Budget Pacing

- Avoid spending the budget too quickly early in the day.
- Aim for a smooth and consistent spending pattern across the entire horizon.
- Avoid spending the budget too quickly early in the day, but also avoid bidding too conservatively when almost no budget has been spent.
- Maintain sufficient participation in auctions to collect informative feedback signals.

Given Parameters

- Campaign Budget: {budget}
- Target CPA: {cpa_target}
- Advertiser Category: {category}, category information may affect traffic quality, conversion efficiency, and bidding aggressiveness.
- Horizon: {horizon} timesteps (a day is split into 48 steps)